RF-One Model Review

Stress-test of the knowledge model, first pass

Prepared by Shelbi Fox | **Date** 17 August 2026 | **Scope** RF-One repository at commit cf577e0, plus the three strategy papers

Before you read this

You asked me to stress-test the model and be ruthless, and to write my objections in red. So that is what this is. I want to say one thing before you start, though, because tone does not travel well in documents like this.

None of what follows is a criticism of the work. I went at it hard because you asked me to, and because the model is good enough to be worth attacking. Nobody bothers stress-testing something that is not going anywhere. There is real thinking in this repository, and I have tried to be equally specific about what is working, not just what is broken.

> **This is a work in progress.** It is a first pass, and I am certain I have misread things. Where I say something is missing, it may simply live somewhere I have not been given access to. Where I say two documents contradict each other, you may have already resolved it in a conversation I was not part of.
>
> I would rather raise something and be wrong than sit on it and be polite. Tell me where I have missed context and I will revise. Treat this as the opening of a conversation, not a verdict.

What I looked at

I read the whole knowledge repository, not a sample of it.

- All 16 **Core** documents - principles, Entity, Goal, Process, Relationship, Corporate, Brand, Operational Unit, Operational Area, Glossary, architecture, implementation, external data mapping, and Core Evolution.
- All 6 **Restaurant Domain** documents - Product, Specification, Ingredient, PurchasingModel, OU-Restaurant, OperationalArea.
- All 16 **Purchasing** documents - business rules, entity definitions, data dictionary, validation, error handling, permissions, acceptance criteria, testing strategy, roadmap, examples.
- All 19 **Commercial Catalog** documents - Catalogue, Version, Publication, Entry, Item, Price, Price List, Tax Category, Availability, Modifiers, Bundle, Offer, Sales Channel and the rest.
- **Sales** (Combo, Clover, Toast), **Environment**, README and PROJECT_STATE.
- The three strategy papers - Goal-First Strategy, Goal-to-Brand-to-Behavior, Selection Intelligence.
- **Old/** - only where the current model still refers to something that no longer exists in it.
- The reference invoices in **Assets/ReferenceDocuments**.

What I did with it

1. **Read it for internal consistency.** Where does the model contradict itself, and are both sides marked Approved?
2. **Read it across layers.** Does every concept sit in exactly one of Core, Domains, Environment, the way the principles require?
3. **Compared the repository against the strategy papers.** Can the model as written actually carry the product the strategy describes?
4. **Tested it against reality.** I took the real supplier invoices and walked them through the Purchasing rules line by line, exactly as written.

The fourth one turned out to be where most of the value was, so I have given it its own section.

What is working

I want to be specific here rather than polite, because these are the parts I would build on.

- **Process as knowledge rather than a flowchart.** Defining a Process to carry rationale, approved and discouraged variants, common mistakes, training notes and verification is genuinely different from how most systems model process, and it is the right instinct.
- **Product plus Specifications equals Ingredient.** Clean, and it solves a real problem. Tomato is a Product; San Marzano, Italian, Organic are Specifications; the combination is the Ingredient. That is elegant.
- **The Catalogue Version and Catalogue Entry architecture.** One Item published differently across locations, channels and dayparts, with no duplication and full version history. This is the strongest engineering in the repository.
- **The immutable Purchase Document.** Never editing the supplier's document, recording anomalies separately in a Validation Log, and treating the original as the legal record. Correct, and it held up when I tested it.
- **Core Evolution.md.** Honestly the healthiest document in the repo. It openly records that the Restaurant domain forced the Core to change. That is a model learning from reality, which is exactly what you said you wanted.

What I would push back on

Ordered by what it would cost to find out late rather than now.

1. Recipe does not exist, and it is holding up everything

Recipe is referenced in twelve documents across Core Domain, Purchasing and Commercial Catalog. It is defined in none of them. It exists only in Old/.

This matters more than a missing file normally would. Purchasing is 16 documents deep and ends at Ingredient cost. The Catalog is 19 documents deep and starts at Item and Price. Neither one ever mentions the other. Purchasing never refers to Item or Catalogue Entry; the Catalog never refers to Ingredient or Supplier Product. Recipe is the bridge, and it is not there.

Practically: right now the model cannot tell me the food cost of the Margherita Pizza. Both halves are documented in detail. There is no path between them. Inventory and Food Cost are in the same position - referenced everywhere, defined nowhere.

2. The Core has no place for a Decision, and the strategy is made of decisions

Entity.md lists what is explicitly not an Entity, and Decision is on that list alongside State and Event. The older Decision.md in Old/ goes further - it calls a Decision transient, "computation, not knowledge," with no identifier, and not normally stored.

Then the Goal-First paper names **Decision Memory** as function five, and section seven calls the growing history of situation, decision, action and outcome the durable advantage that competitors cannot copy.

So the one asset the strategy identifies as the moat is the exact thing the Core refuses to persist. You cannot accumulate or learn from a class of object your model defines as transient and identity-less. And I do not think "that is runtime, not Core" resolves it, because Principle 15 says facts are persisted and conclusions are inferred. A decision that was taken, and what happened next, is a fact.

3. Goal-First is not possible under the Core's own definition of Goal

Goal.md says a Goal exists only once a Process has been defined to pursue it, and that a desired outcome without a Process is a **Desire**, not a Goal.

Every strategy paper starts from client goals before any process exists. By the Core's own definition those are all Desires. And Desire appears exactly once in the entire current repository - inside that one sentence in Goal.md - and is defined nowhere. It was a full document in Old/ and did not survive the restructure.

This is my single most useful question for you, and it is cheap to answer: **were Decision, Desire, Recipe and Vision dropped deliberately, or lost when the structure was reorganised?** If deliberately, I am arguing with a real position. If not, I am just handing you bugs. It changes how I write the rest of this up.

4. Item and Ingredient are two unreconciled models of the same thing

Item.md calls itself the single source of truth for every commercial entity. Product.md calls itself the canonical vocabulary of the Restaurant Domain. Both list Olive Oil as an example. Item.md routes Item through to Purchasing, while Product.md says Products are never purchased directly and Purchasing routes everything through Supplier Product to Ingredient. Two contradictory purchasing paths, both marked Approved.

5. Normalising everything to grams

Rule 7 requires every purchasable Ingredient to be held in grams and Rule 8 requires cost per gram. This works for flour and cheese. It does not work for wine, beer, spirits, countable goods like eggs, or non-food lines like delivery charges and napkins - all of which arrive on the same invoices. The invoice test below is really a test of this rule.

The fix already exists inside your own model. UnitOfMeasure.md defines Mass, Volume, Quantity and Time as measurement types. Purchasing simply never adopted it.

6. Files that look finished but are empty

Worth knowing because they suppress the question rather than raising it:

- **Sales/Clover** - all eleven files are zero bytes, including CloverAPIAnalysis, CloverDataMapping, Orders, Payments and KnownLimitations.
- **Domain/Ingredient.md** - reads "See generated content placeholder." This is the single most referenced entity in the Restaurant domain.

- **Purchasing/Workflow.md** - contains only "Step 5." Steps 1 to 4 and 6 onward are not there.
- **Menu.md, ServiceSequence.md, Sales/Toast/README.md** - all zero bytes.
- **Environment** - three lines, and it is one of the three top-level layers. Tax and regulatory work has nowhere to live until it exists.

Menu is the one I would treat as urgent rather than untidy, given that Mount Dora opens shortly and each location runs its own menu.

The invoice test

This is the part I would read first if I were you, because it is the only part where the model meets something real.

I took Invoice 6855 from Assets - BBC Wine Imports to Rome's Flavours Winter Park, dated 15 July, seven lines, a completely ordinary delivery - and walked it through the Purchasing rules exactly as written.

> **Result: the module can fully process none of the seven lines.**

Not because the invoice is strange. It fails because all seven lines are **resale goods** - wine and beer bought to be sold, not cooked with. Rule 4 requires every Supplier Product to map to exactly one Ingredient, and an Ingredient is defined as a culinary entity that supports Recipes. A bottle of Amarone is not an ingredient. It is an Item, and Item.md even lists House Wine as one of its own examples. The module has no route for goods that are resold, and Rule 4 allows no exception.

Then Rule 7 asks me to convert it to grams. Here is the same cost expressed three ways:

Amarone della Valpolicella 2018, 12 x 750ml, $648.00	Value	Useful?
Cost per bottle	$54.00	Yes - how it is bought and sold
Cost per 5oz pour	about $10.65	Yes - how margin is set by the glass
Cost per gram, as Rule 8 requires	**$0.0727**	**No**

To reach that last number I had to assume a density for wine, which varies with alcohol and sugar. So the rule introduces an estimate in order to destroy a figure that was already exact. There is also a sequencing problem: density is stored on the Ingredient, but Rule 4 lets a new Supplier Product stay unmapped for a while, so on first receipt there is no density available and every line stops for human review.

I then checked two of the photographed invoices for contrast

Samuels Seafood normalises perfectly. It has a proper unit column, and pounds convert to grams exactly. That is the case Rule 7 was designed for, and it works. But its third line is a **$2.99 delivery charge**, entered as a line item rather than a document-level charge. Rules 3 and 4 then require that delivery charge to become a Supplier Product mapped to an Ingredient, and Rule 7 requires it to be expressed in grams.

Cheney Brothers prices by the case with a separate weight column, and labels its total "Estimated Total" - normal for catch-weight goods, which get restated after weighing. The model has no concept of a document that will later be corrected, which sits awkwardly with the rule that documents are immutable.

Three suppliers, one restaurant, one month, and three different meanings for the quantity column:

Supplier	Quantity means	Unit column	Extra charges
BBC Wine	bottles (inferred)	none	3% card fee, conditional
Samuels Seafood	pounds or portions	LBS / LB / EA	delivery as a line item
Cheney Brothers	cases	CS	fuel surcharge (handled correctly)

The one that worries me most

The wine invoice has no unit column at all. Reading the arithmetic, quantity has to mean bottles. But nothing on the document says so. If a mapper reads line 1 as twelve cases rather than twelve bottles, the volume goes from 18 litres to 108 litres and the cost per gram is understated **six times over**. The salmon line has the same ambiguity at two times, because six pounds and six eight-ounce portions produce the identical line total.

Neither error trips any validation rule. ValidationRules.md catches an impossible conversion, and both of these are entirely possible - just wrong. Silent errors that pass validation worry me more than loud ones that fail.

Fields the invoices carry that the model cannot store

Payment terms, due date, delivery date (which differs from invoice date), supplier address and contact details, sales rep, the minus $57.61 payment line, and the Florida beverage licence number on the bill-to. That last one is the identifier that makes an alcohol purchase lawful, and there is nowhere to put it. It belongs in Environment, which is currently three lines.

There is also a $1,452.00 total and a $1,394.39 balance due, and only one TotalAmount field. Whichever one gets mapped, roughly four percent goes missing somewhere.

Questions I could not answer from the documents

1. Were **Decision, Desire, Recipe and Vision** dropped deliberately, or lost in the restructure? This is the one I would most like answered first.
2. Is the Core **frozen or evolving**? Core Principles is marked Core 1.0 Freeze and calls the principles immutable. Core Evolution says a mature Core is the result of evolution, not the starting point, and that domains should never be forced to fit an inadequate Core. Both are Approved. This one gates everything else, because if the Core is frozen then none of the above can be acted on.
3. Is the **Commercial Catalog** a Restaurant concept or a shared one? All nineteen files say industry independent and give retail examples, but the most recent commit deliberately moved it under Restaurant.
4. Are **Item and Ingredient** the same thing, siblings, or layered? No document reconciles them.
5. **What would tell you the Core is ready to build on?** There are 120 documents and no code yet, and I could not find a stated criterion for when modelling stops and building starts. Not a criticism - I genuinely could not find it, and I think it is the most important thing on this page.

What I would suggest next

1. **Rewrite Rule 7** to normalise to the canonical unit of the measurement type - mass, volume or count - reusing UnitOfMeasure.md. Keeps the intent, removes the largest single source of failure.
2. **Give Purchase Line a type** - goods, service, or charge - so a delivery fee is never forced to become food.

3. **Add a resale path** so wine, beer and merchandise can be purchased as Items without pretending to be Ingredients.
4. **Write a quantity rule.** The six-times and two-times errors above are the highest-risk defects I found, precisely because they pass validation.
5. **Define Recipe**, even minimally. Until it exists, Purchasing produces knowledge nothing can use.
6. **Run Mount Dora through the model on paper before it opens.** A second location exercises multi-location, per-location menus and pricing all at once. It is the best test available and the timing is right in front of us.

Where this leaves us

Short version: the architecture is sound, the Purchasing rules were written for dry goods and need widening, and the chain from what you buy to what you sell has a missing link in the middle. None of that is fatal and most of it is cheaper to fix now than after code exists.

The thing I keep coming back to is that there is a real restaurant trading a few miles away, generating exactly the data that would settle half of these questions, and the model has never been pointed at it. One invoice produced everything in the section above. Ten would probably produce a finished set of rules.

Happy to go deeper on any of this, and equally happy to be told I have got the wrong end of it. Tell me which parts you want me to take further.

Shelbi Fox | 17 August 2026 | First pass, prepared for discussion